

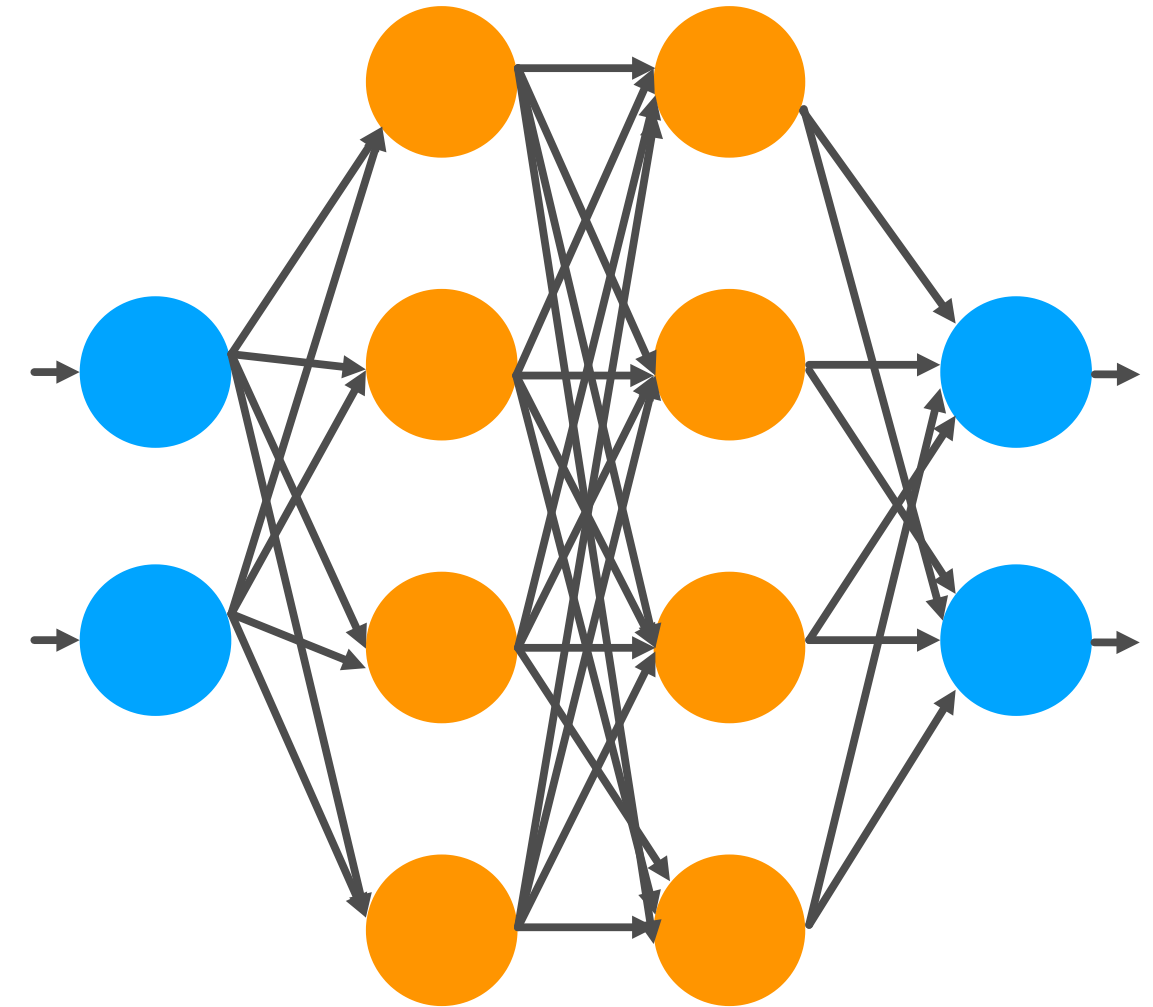


NEURAL NETWORK

ว30263

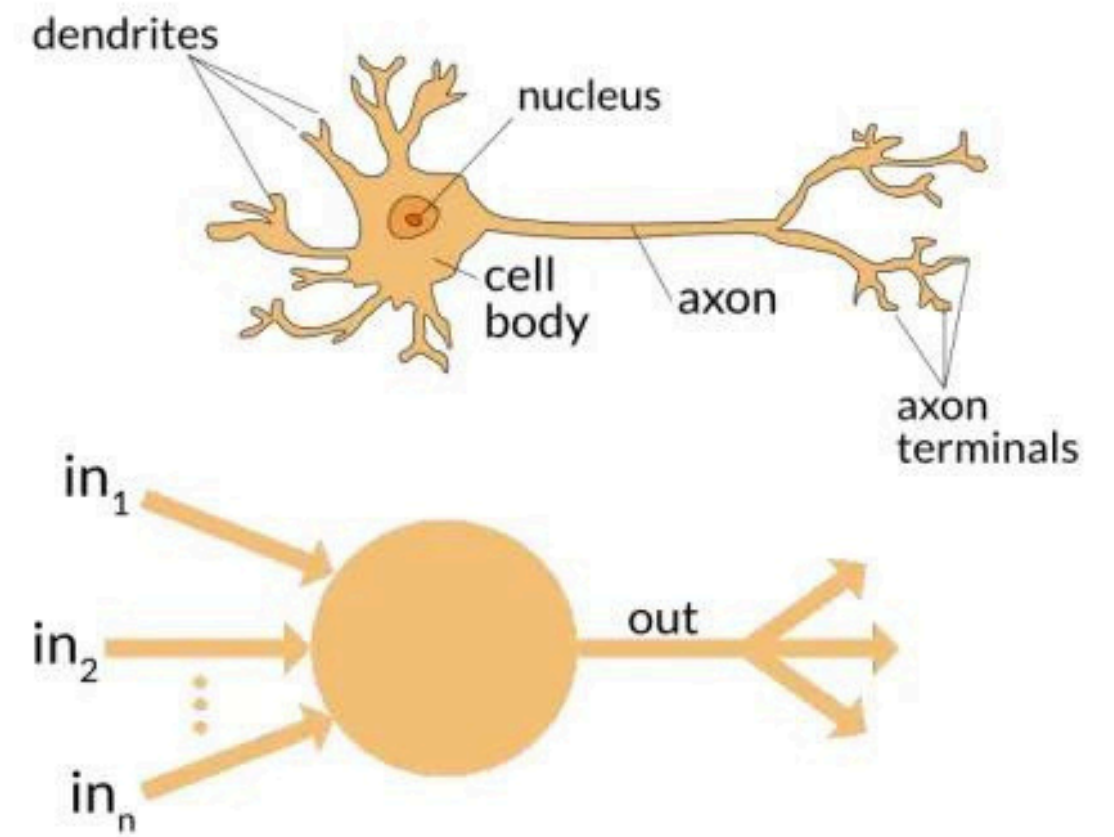
การเขียนโปรแกรมปัญญาประดิษฐ์

WHAT IS NEURAL NETWORK?



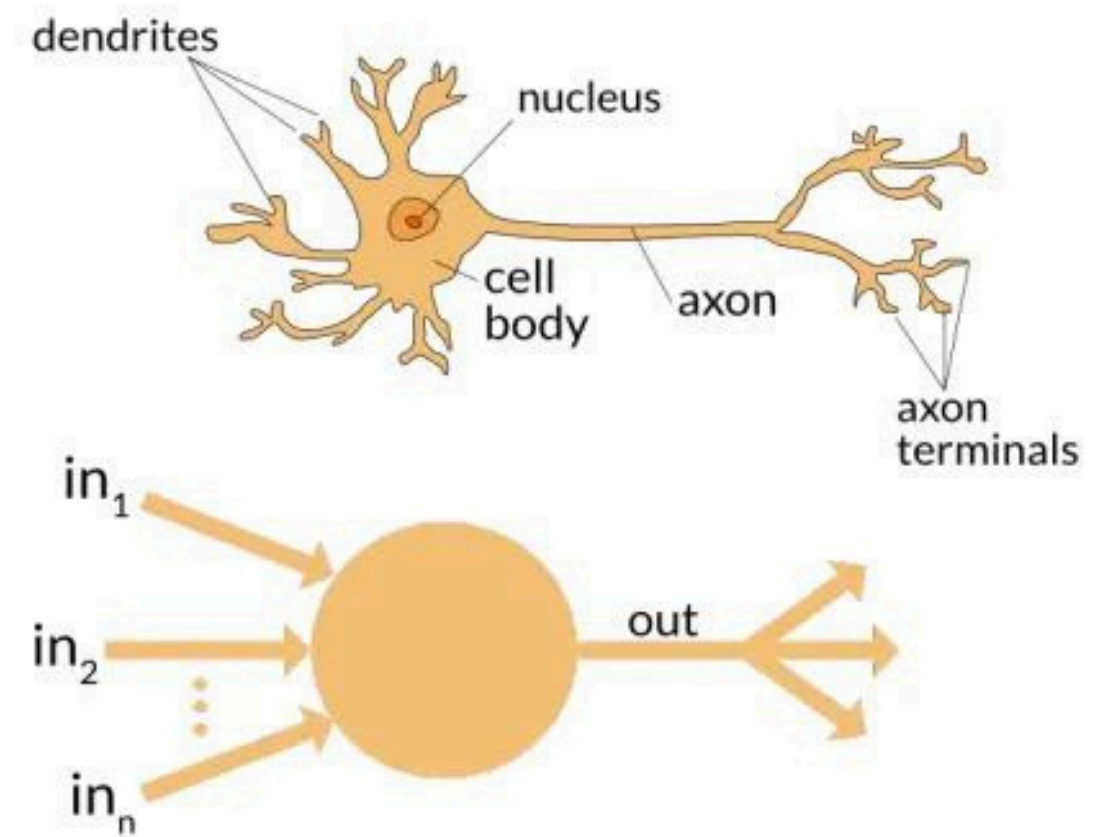
โครงข่าย ประสาทเทียม (NEURAL NETWORK)

โครงข่ายประสาทเทียม (Neural Network) เป็นสาขาหนึ่งของปัญญาประดิษฐ์ Artificial Intelligence (AI) เป็นแนวคิดที่ออกแบบระบบโครงข่ายคอมพิวเตอร์



โครงข่าย ประสาทเทียม (NEURAL NETWORK)

ให้เลียนแบบการทำงานของสมองมนุษย์
มนุษย์มีกระบวนการเรียนรู้โดยจดจำรูปแบบ คือ มนุษย์สังเกตุดูเห็นสิ่งของที่ไม่รู้จัก
เมื่อเกิดการเห็นบ่อยๆมากขึ้น เห็นในรูปแบบที่แตกต่างมากขึ้นก็จะเกิดการเรียนรู้
และจดจำในสิ่งที่คล้ายๆกัน

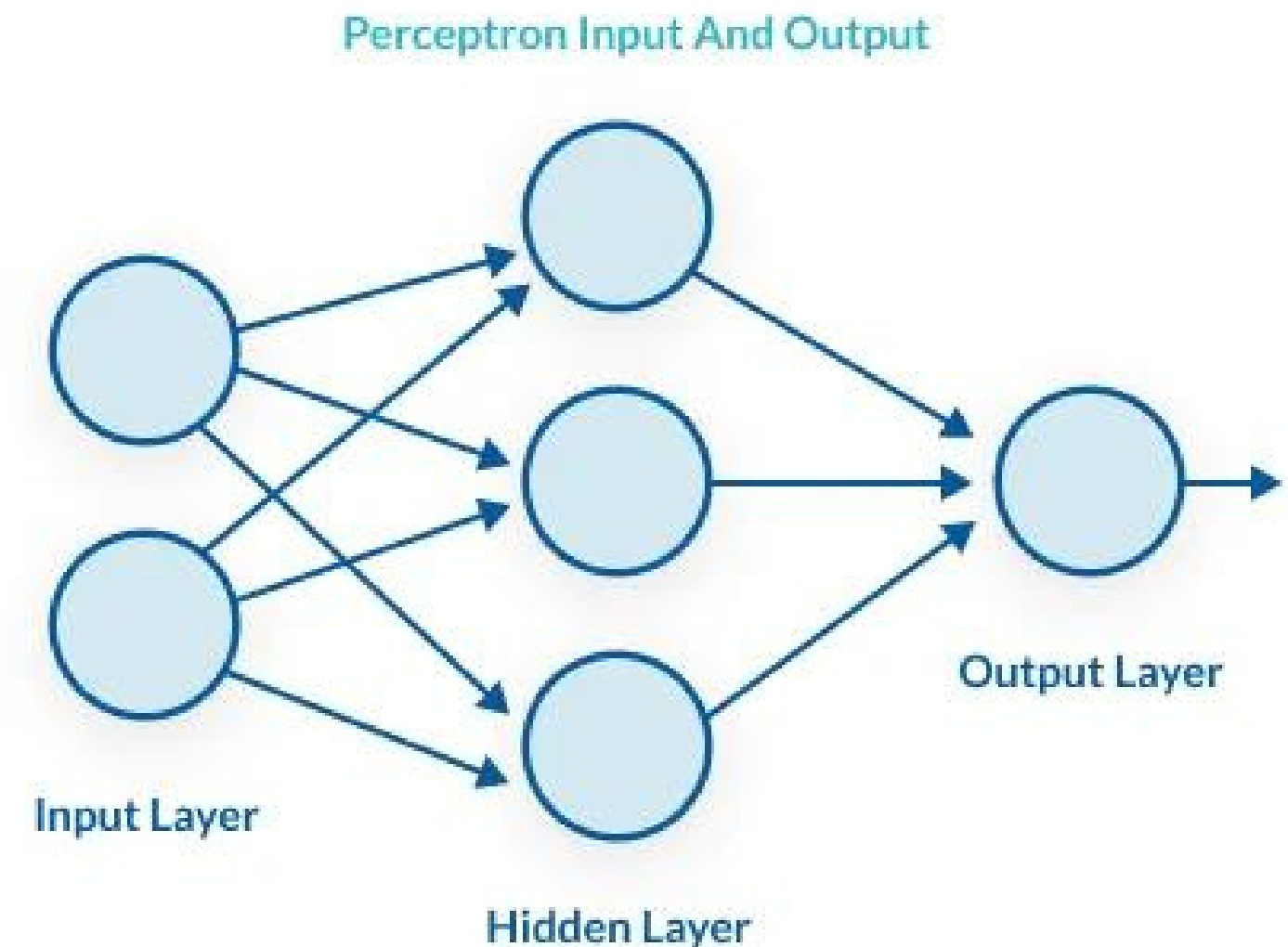


WHAT IS MLP?



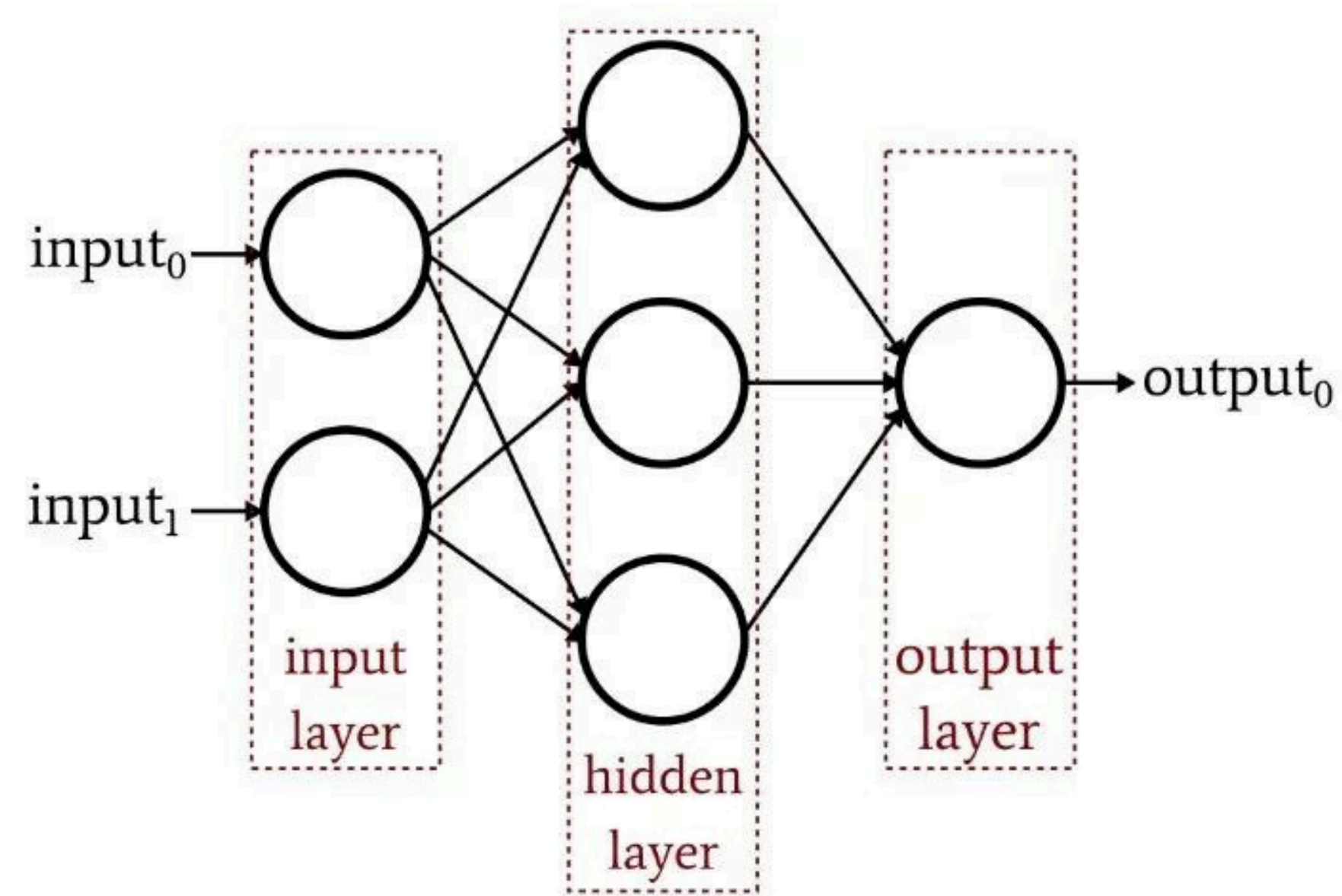
MULTI-LAYER PERCEPTRON (MLP)

Neural Network จะแบ่ง Perceptron ออกเป็นชั้น โดยแต่ละชั้นจะเรียกเป็น Layer โดยข้อมูลที่เข้ามาจะไหลไปในทิศทางเดียว ไม่ไหลย้อนกลับจาก Layer หนึ่งสู่อีก Layer หนึ่ง



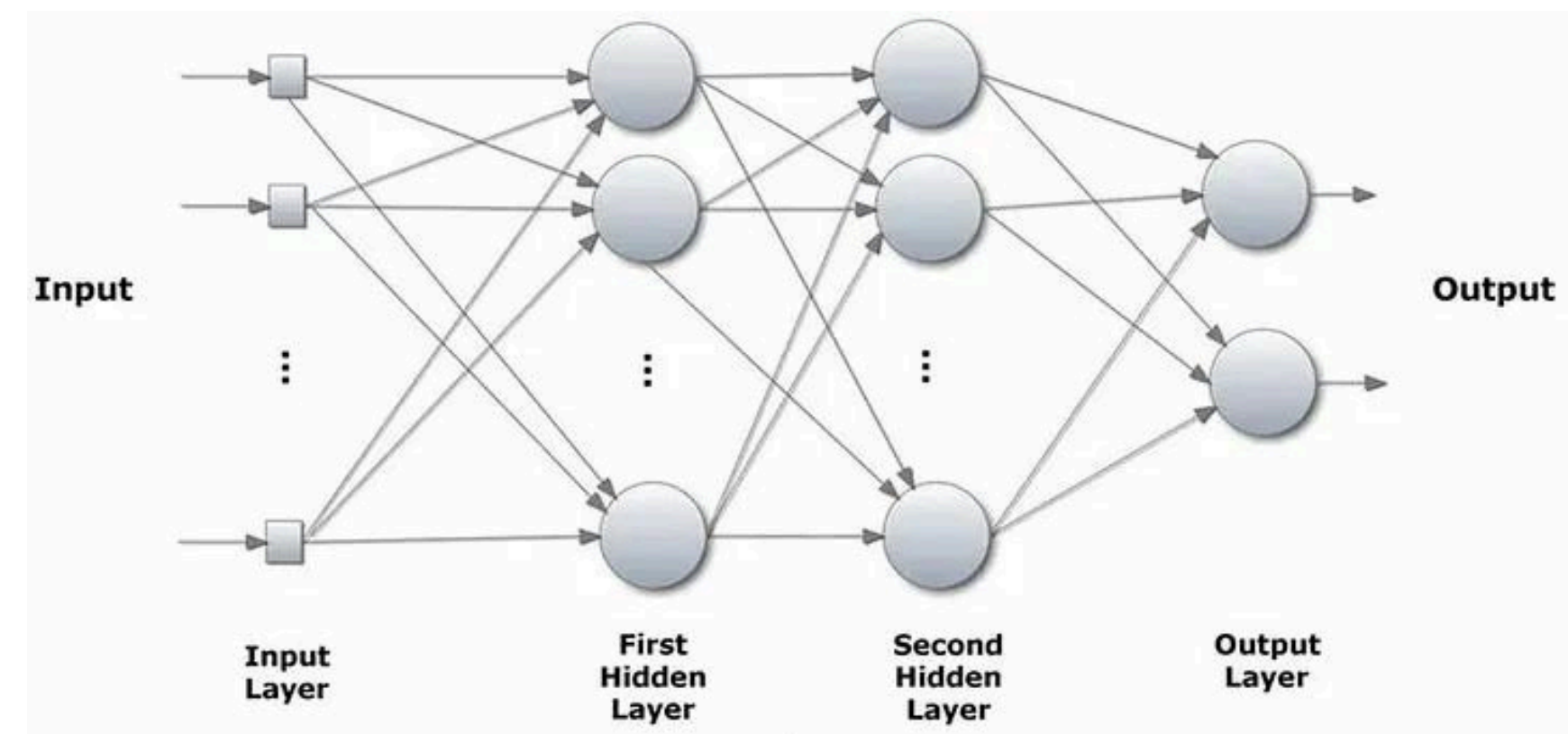
MULTI-LAYER PERCEPTRON (MLP)

Neural Network แบบเพอร์เซ็ปตรอนหลายชั้น(Multi-Layer Perceptron : MLP)จะประสานการทำงานผ่านส่วนที่เรียกว่าโหนด (Node) ประกอบไปด้วย 3 ชั้นมีการเชื่อมต่อการทำงานของโหนดอย่างสมบูรณ์ (Fully-Connected)



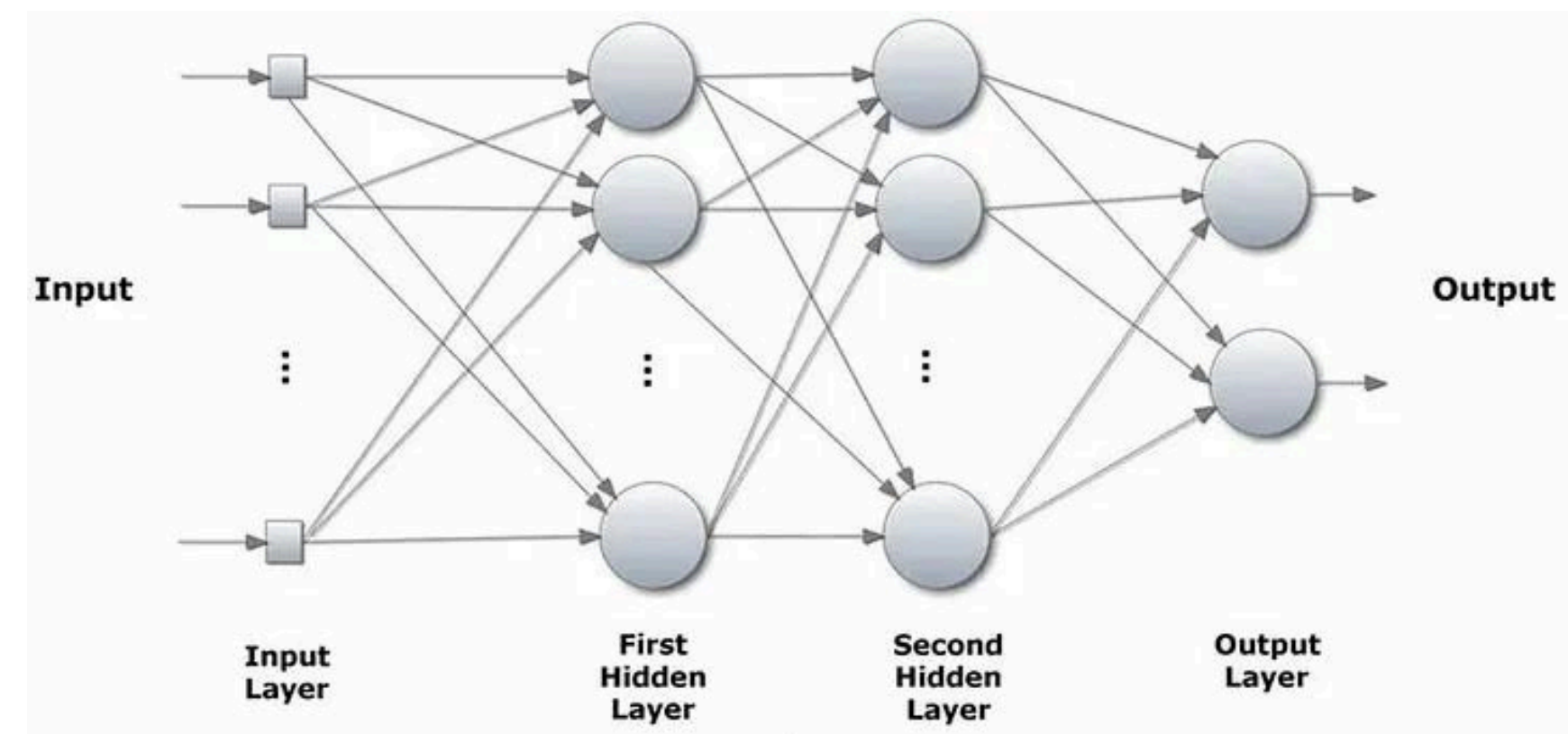
MULTI-LAYER PERCEPTRON (MLP)

Input Layer ชั้นนี้จะเป็นส่วนที่จัดการข้อมูล input จำนวนของโหนดขึ้นอยู่กับจำนวนของ input ว่าข้อมูลอะไรบ้างที่จะนำเข้ามาคิดในโมเดล



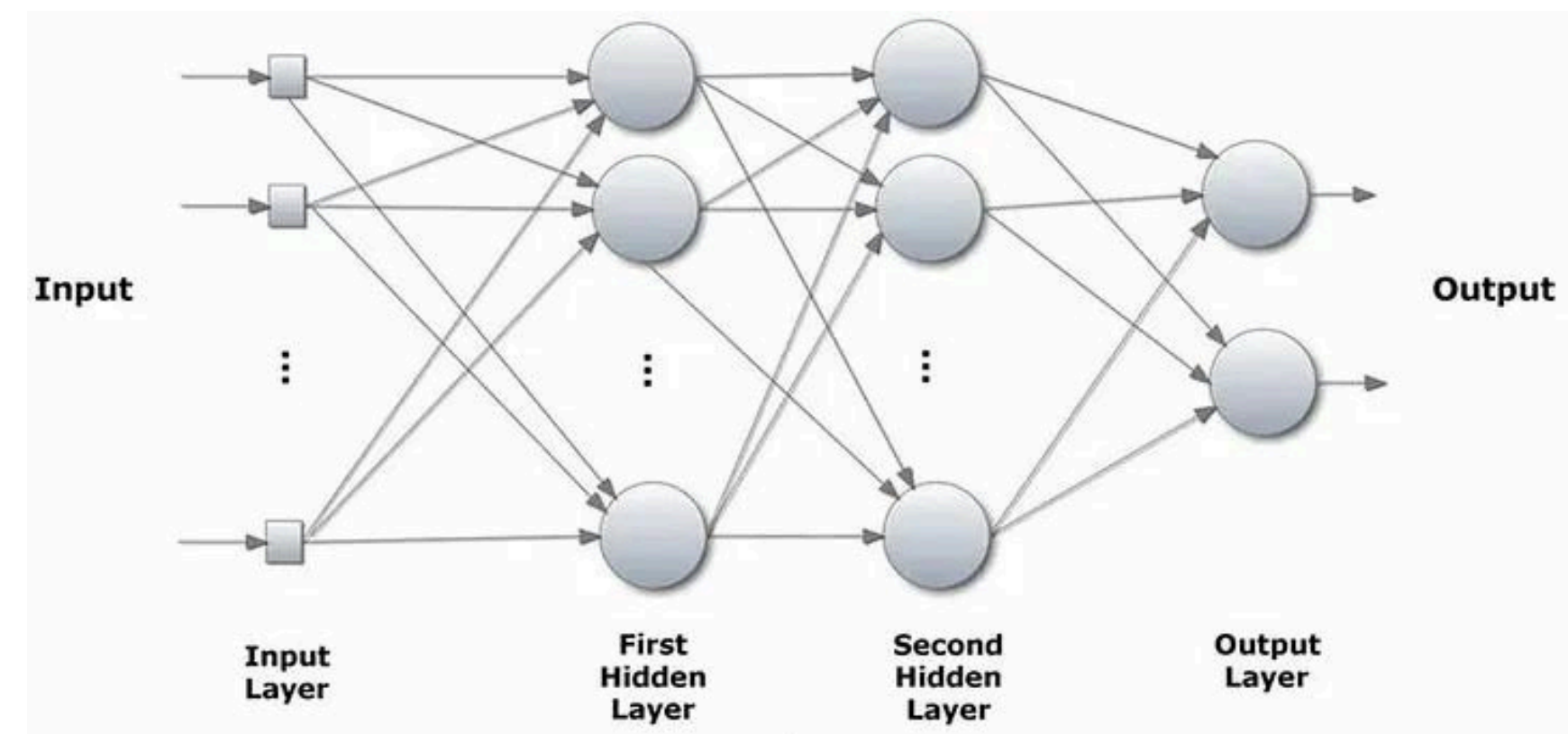
MULTI-LAYER PERCEPTRON (MLP)

Hidden Layer ชั้นที่อยู่ตรงกลางมีผลต่อประสิทธิภาพในการเรียนรู้ของโมเดล ซึ่งชั้นนี้จะมีกี่ชั้นก็ได้ มีจำนวนของ Neuron เท่าไหร่ก็ได้ การเพิ่มชั้นและจำนวน neuron ก็จะมีส่งผลต่อการทำงานของโมเดลมากๆ มีการเรียนรู้ข้อมูลในเชิงลึกเรียกว่า Deep Learning

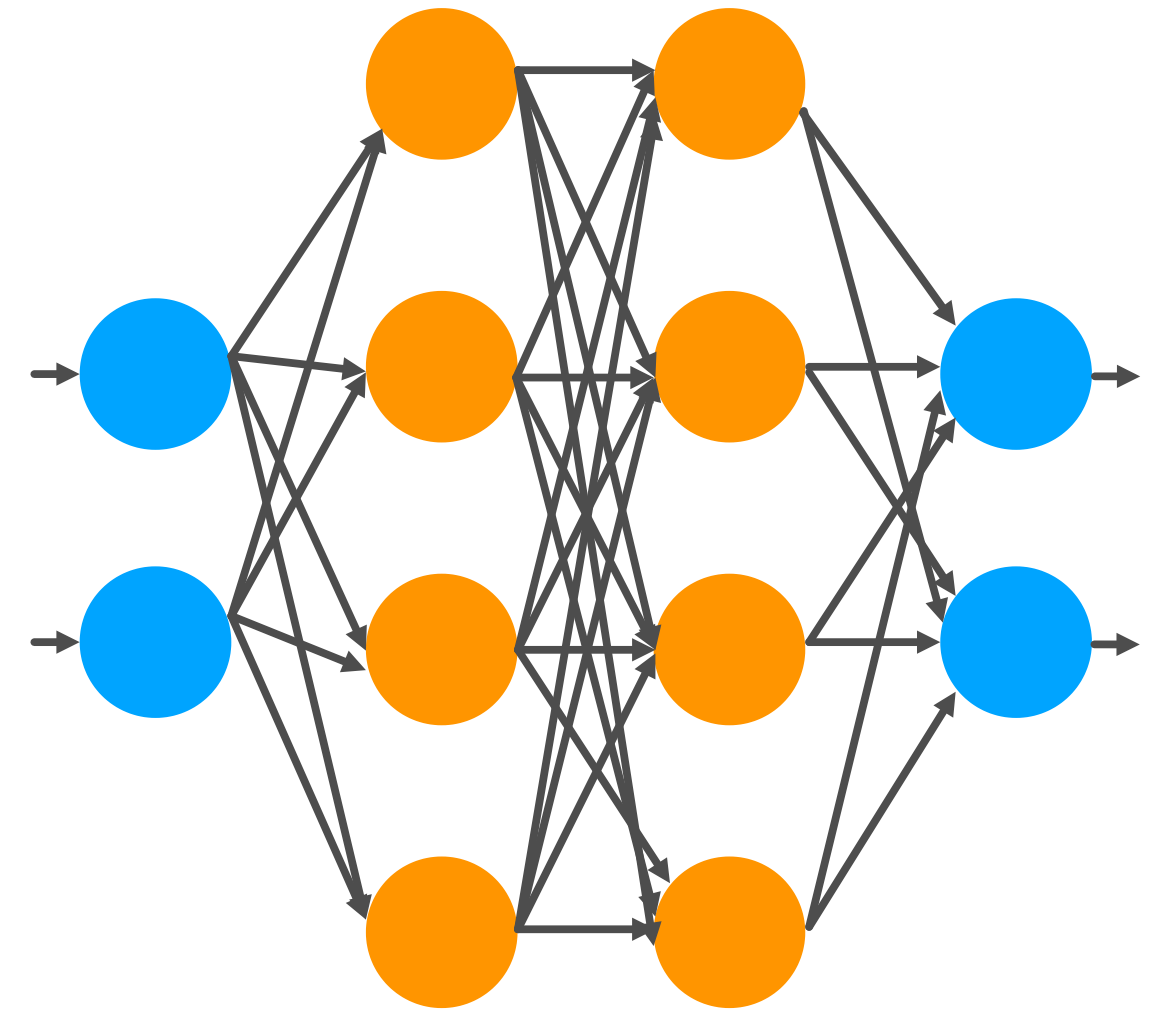


MULTI-LAYER PERCEPTRON (MLP)

Output Layer ชั้นนี้จะนำเอาข้อมูลจากการคำนวณใน Hidden Layer ไปใช้งานจำนวนของโหนดที่ส่งออกมาในชั้นนี้ขึ้นอยู่กับรูปแบบของ output ที่จะเอาไปใช้

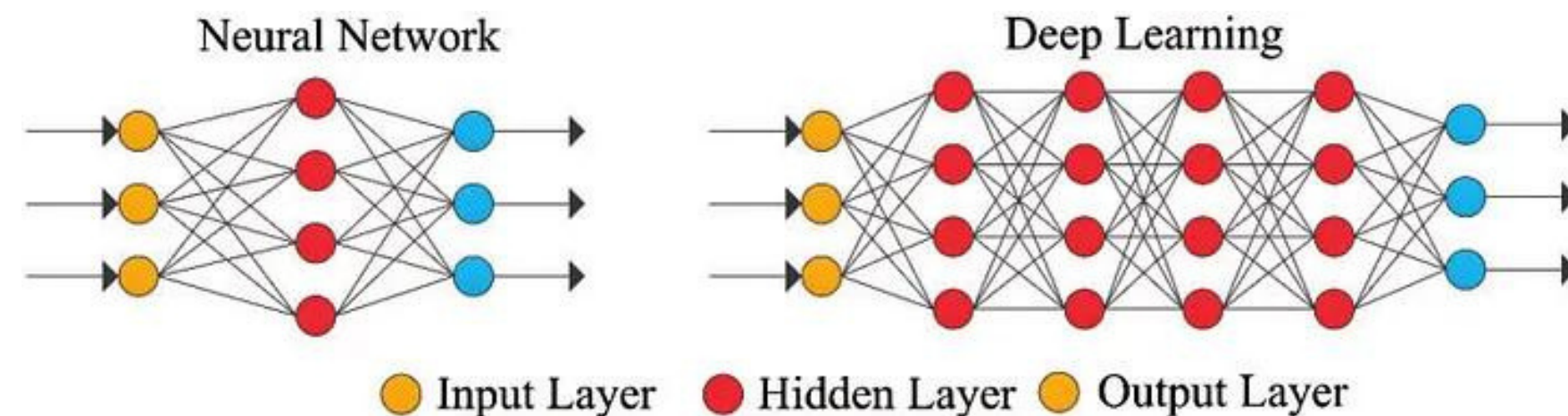


**ความแตกต่าง
และความ
เหมือนกัน
ระหว่าง NEURAL
NETWORK และ
DEEP LEARNING**

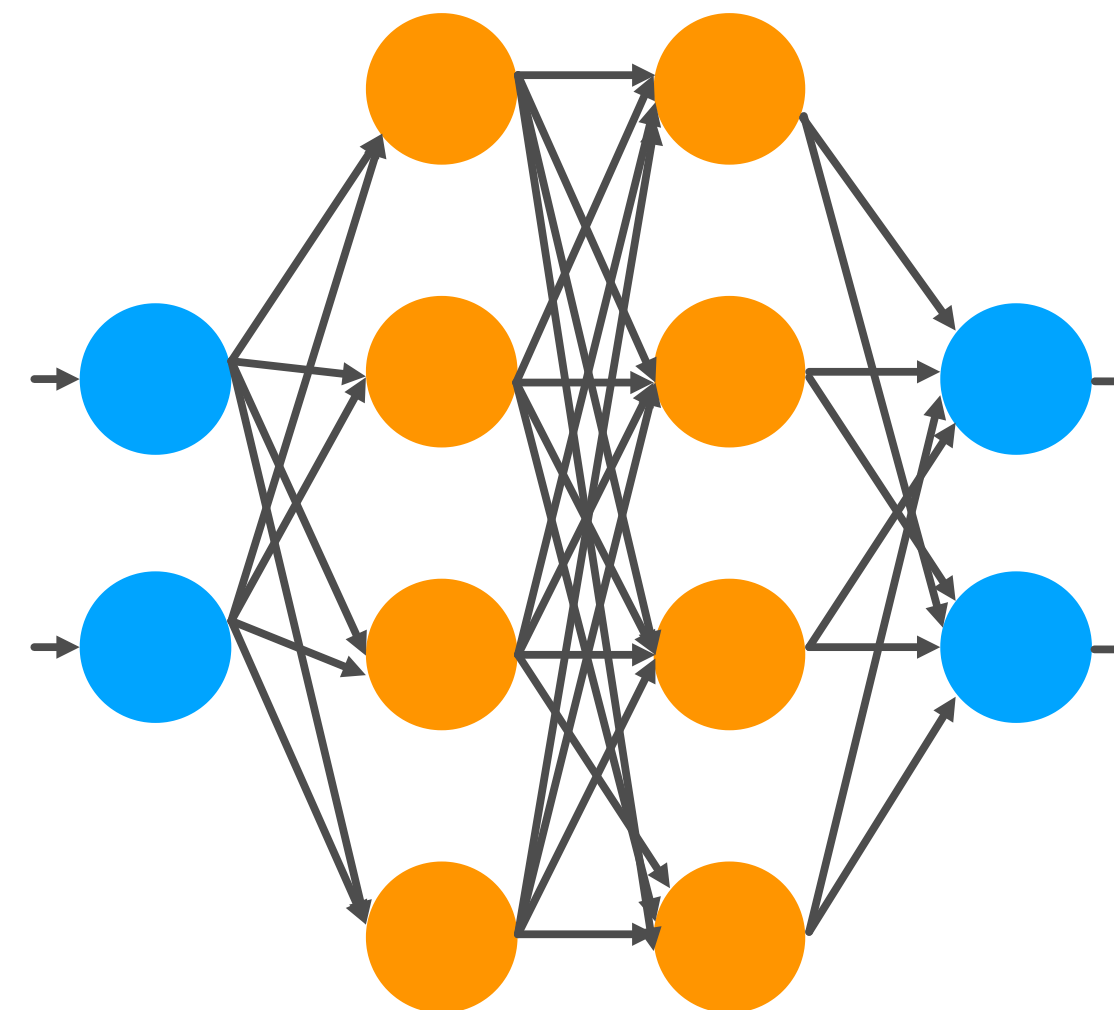


MULTI-LAYER PERCEPTRON (MLP)

สมองของมนุษย์เรานั้นก็เปรียบเหมือน Hidden Layer ที่เชื่อมต่อระหว่าง Input Layer และ Output Layer นั้นเอง แต่ในชีวิตจริงสมองของมนุษย์เรานั้นซับซ้อนกว่าที่เห็นในภาพนี้มาก เพื่อที่จะทำให้ Neural Network ของเรานั้นสามารถคิดและประมวลผลซับซ้อนได้เหมือนสมองมนุษย์ ชั้นที่เป็น Hidden Layer จึงต้องมีหลายๆชั้น ให้มันส่งข้อมูลประมวลผลต่อกันไป ทำให้มันสามารถคำนวณอะไรที่ซับซ้อนได้เยอะขึ้น ซึ่งเราเรียกว่า Deep Learning

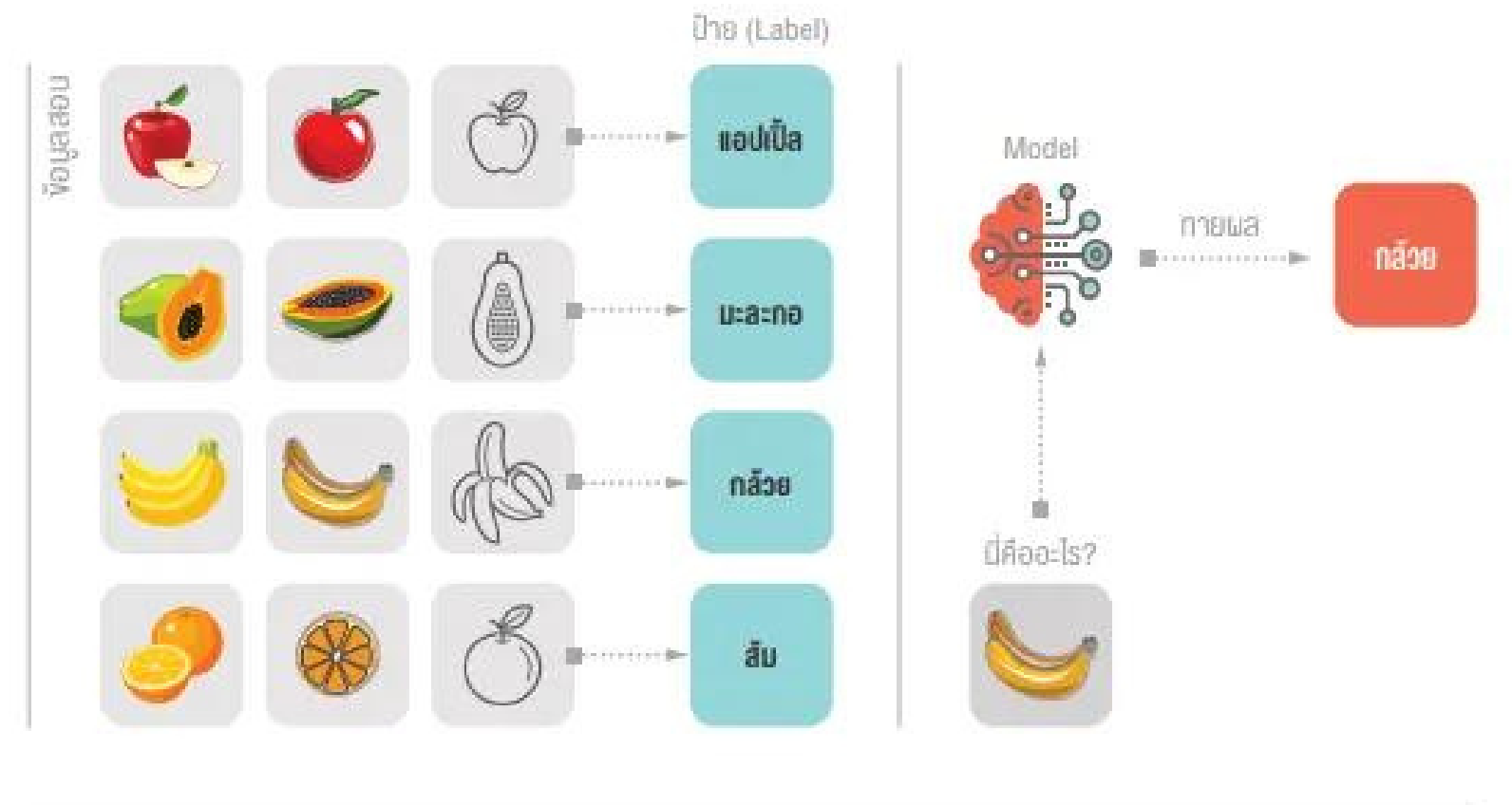


**MACHINE LEARNING คือ
การใช้ 'ข้อมูล' สอนให้
ระบบคอมพิวเตอร์
ทำการเรียนรู้ได้ด้วย
ตนเองแบ่งออกเป็น**



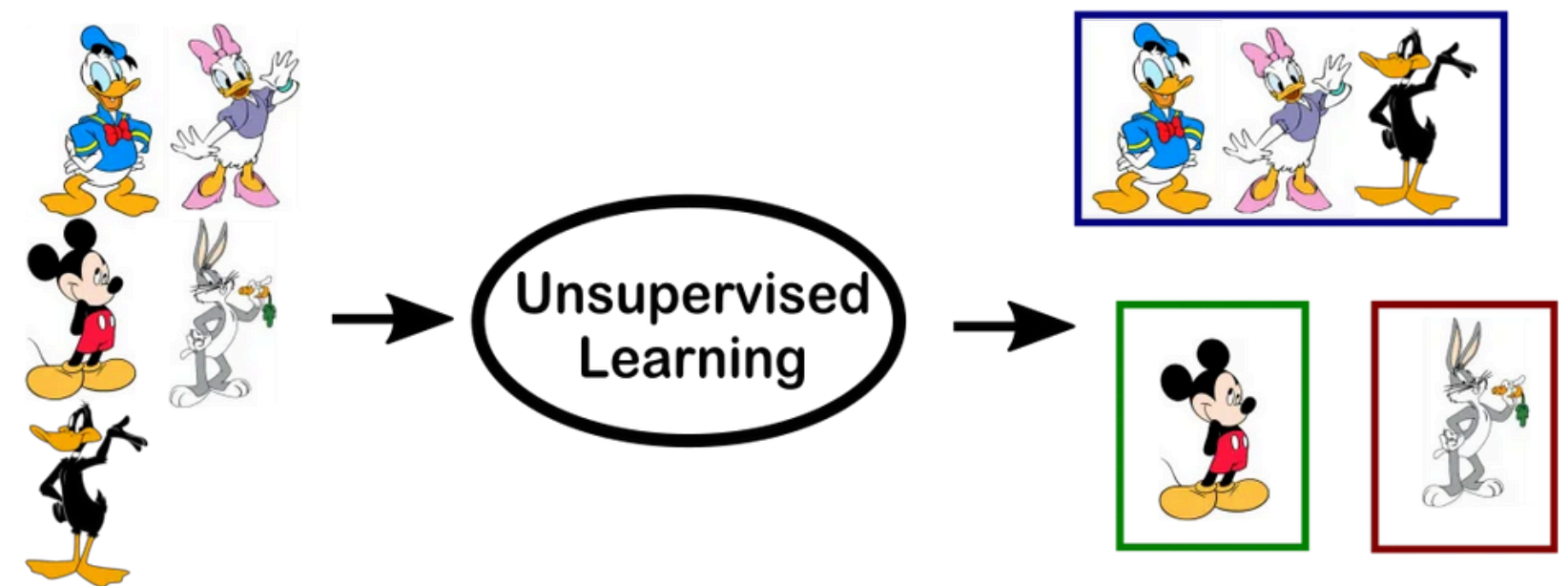
MULTI-LAYER PERCEPTRON (MLP)

การเรียนรู้แบบมีผู้สอน (Supervised Machine Learning Algorithms) ข้อมูลที่ใช้ในการฝึกนั้นเรียกว่า ข้อมูลชุดเรียนรู้ (Training Data / Training Set) นำมาแยกประเภทผลลัพธ์ด้วยการติดป้ายกำกับ (Labels/Class) เป็นผลเฉลย จากนั้นนำข้อมูลที่ติดป้ายแล้วไปใช้ในการฝึกของเครื่องที่ทำงานผ่านอัลกอริทึมสำหรับสร้างโมเดลที่ใช้ในการทำนายผลลัพธ์ โดยนำข้อมูลใหม่หรือข้อมูลชุดทดสอบ (Test Set) ไปใช้ทดสอบประสิทธิภาพของโมเดล



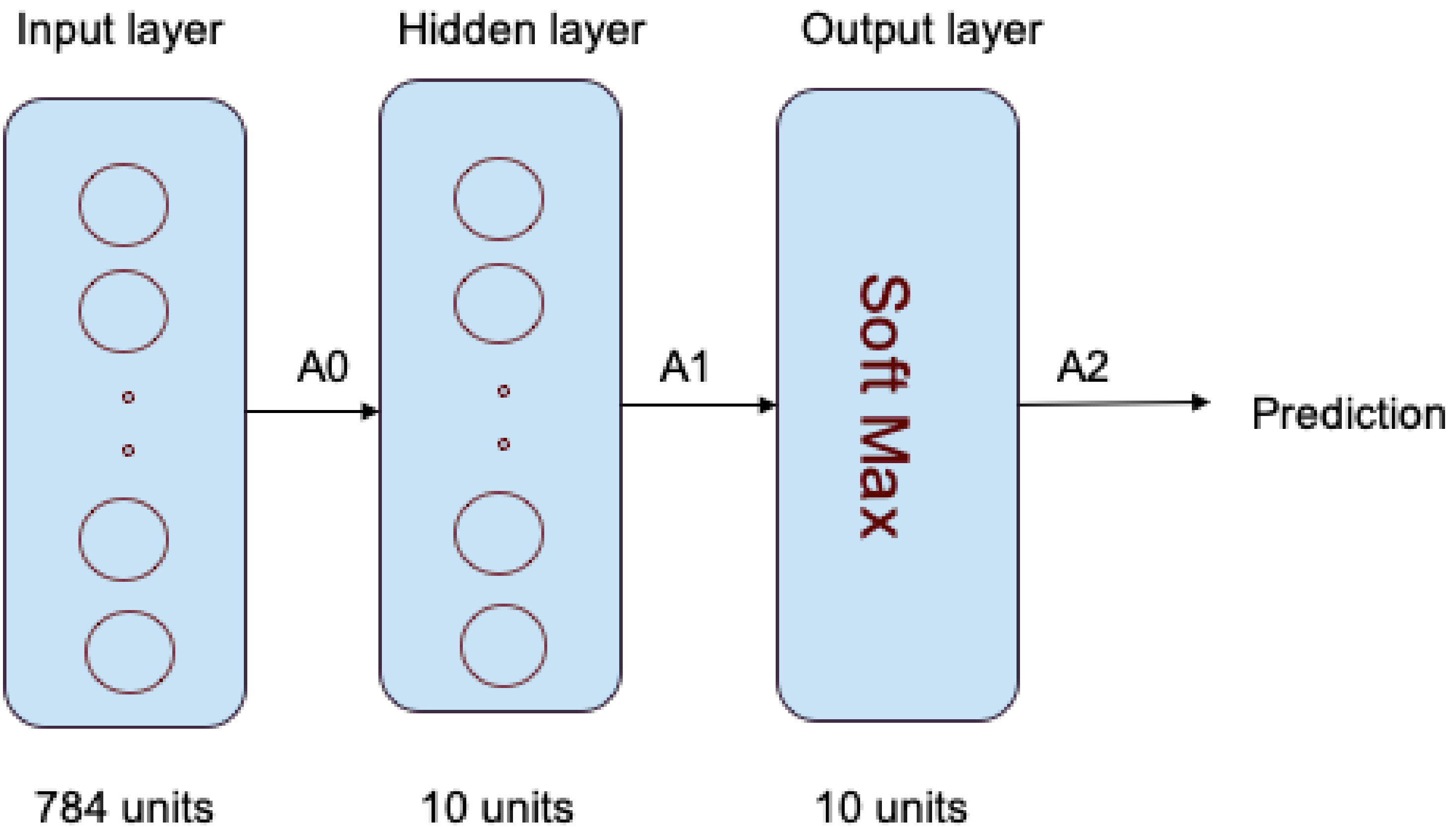
MULTI-LAYER PERCEPTRON (MLP)

การเรียนรู้แบบไม่มีผู้สอน (Unsupervised Machine Learning Algorithms) เป็นการเรียนรู้ด้วยข้อมูลที่ไม่ถูกจัดประเภทหรือติดป้ายกำกับข้อมูล วิธีนี้เครื่องจะคาดเดาข้อมูลที่ได้รับและทำความเข้าใจถึงโครงสร้างที่ซ่อนอยู่ไม่สามารถหาผลลัพธ์ที่ถูกต้องได้ แต่จะใช้วิธี สืบรวจข้อมูลและใช้การประมาณการว่าข้อมูลนั้นคืออะไร





www.cybercontrols.org



ACTIVATION FUNCTION!

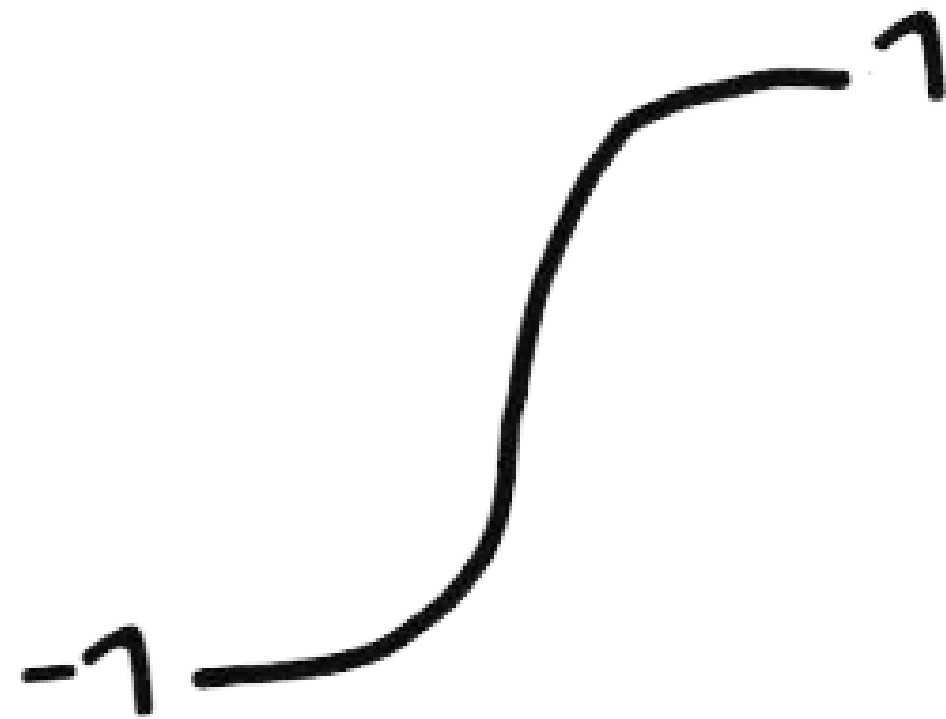
ReLU



Sigmoid



Tanh

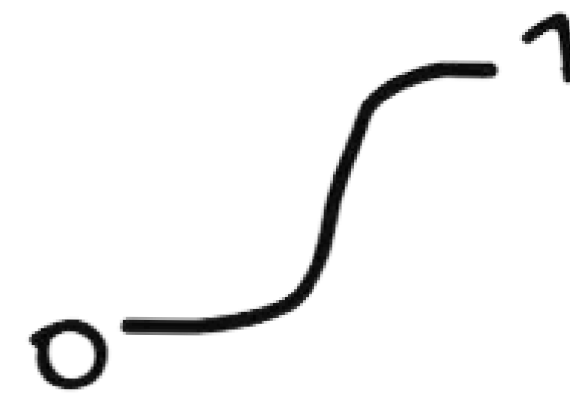


ACTIVATION FUNCTION

activation functions เพื่อบีบตัว output ให้อยู่ในช่วงที่ต้องการ เช่น $[0, 1]$, $[-1, +1]$ เป็นต้น การใช้ weighted sum ร่วมกับ activation functions ช่วยให้ neuron สามารถเรียนรู้ความสัมพันธ์แบบ non-linear ได้ดียิ่งขึ้น

Activation functions ที่เรานิยมใช้ใน neural networks มีอยู่หลายตัว เช่น ReLU, Sigmoid, Tanh, Leaky ReLU, Step, Linear เป็นต้น แต่สามตัวที่ใช้บ่อยสุดอยู่ในรูป

Sigmoid

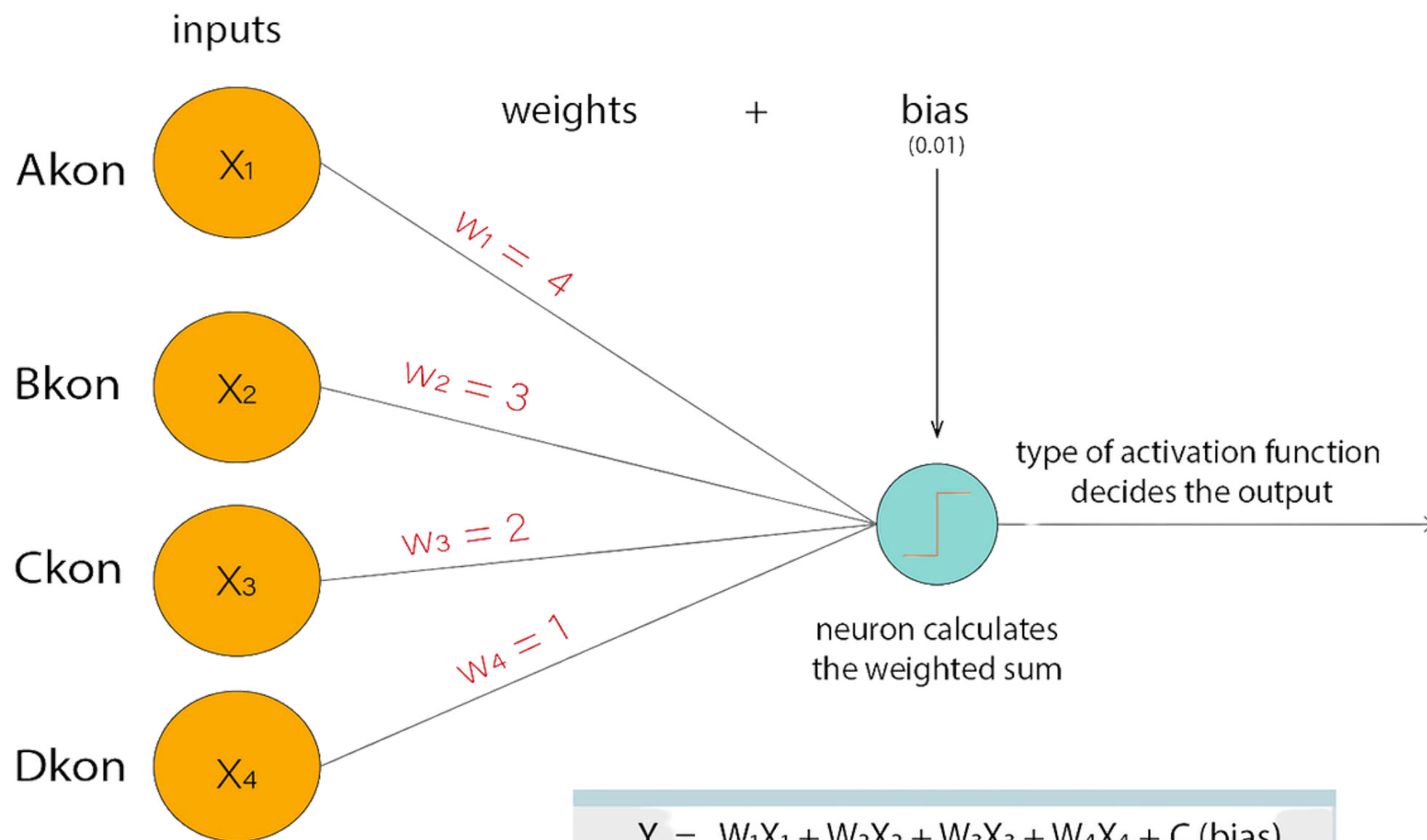


ReLU

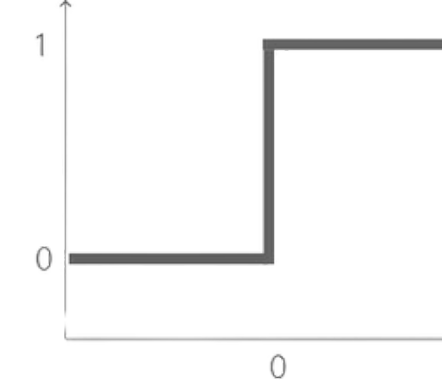


Tanh



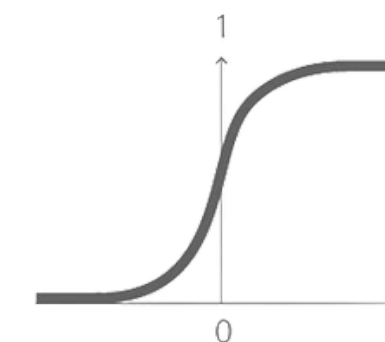


$Y = W_1X_1 + W_2X_2 + W_3X_3 + W_4X_4 + C \text{ (bias)}$			
Thailand	$(4 \times -9) + (3 \times 10) + (2 \times 3) + (1 \times -2)$	=	-2
Jamaica	$(4 \times -4) + (3 \times 5) + (2 \times -1) + (1 \times 8)$	=	5
Dubai	$(4 \times -3) + (3 \times 6) + (2 \times -2) + (1 \times 6)$	=	8



Step

Thailand: 0
Jamaica: 1
Dubai: 1



Sigmoid

(Probabilities)

Thailand: 0.002
Jamaica: 0.85
Dubai: 0.94

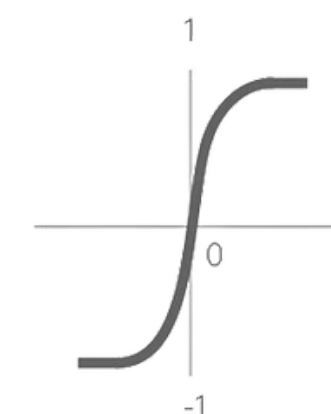
*probabilities do not add to 1

Softmax

(Probabilities)

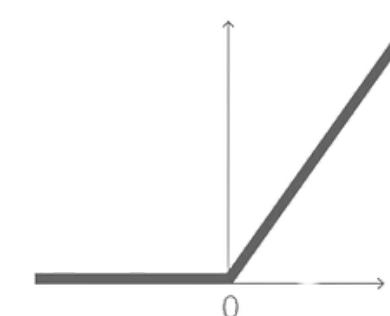
→ Thailand: 0.02
Jamaica: 0.22
Dubai: 0.76

*probabilities add to 1



tanh

Thailand: -0.2
Jamaica: 0.5
Dubai: 0.8



ReLU

Thailand: 0
Jamaica: 5
Dubai: 8

SIGMOID FUNCTION

Sigmoid Function เป็นฟังก์ชันที่เป็น Curve รูปตัว S เห็นแล้วเข้าใจได้ง่าย และเนื่องจาก Output ของ Sigmoid Function มีค่าระหว่าง (0-1) จึงเหมาะที่จะถูกใช้ในงานที่ต้องการ Output เป็นความน่าจะเป็น (Probability) หรือใช้เป็น Output ว่า 1=Yes, 0=No

$$f_{w,b}(x) = g(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

Sigmoid

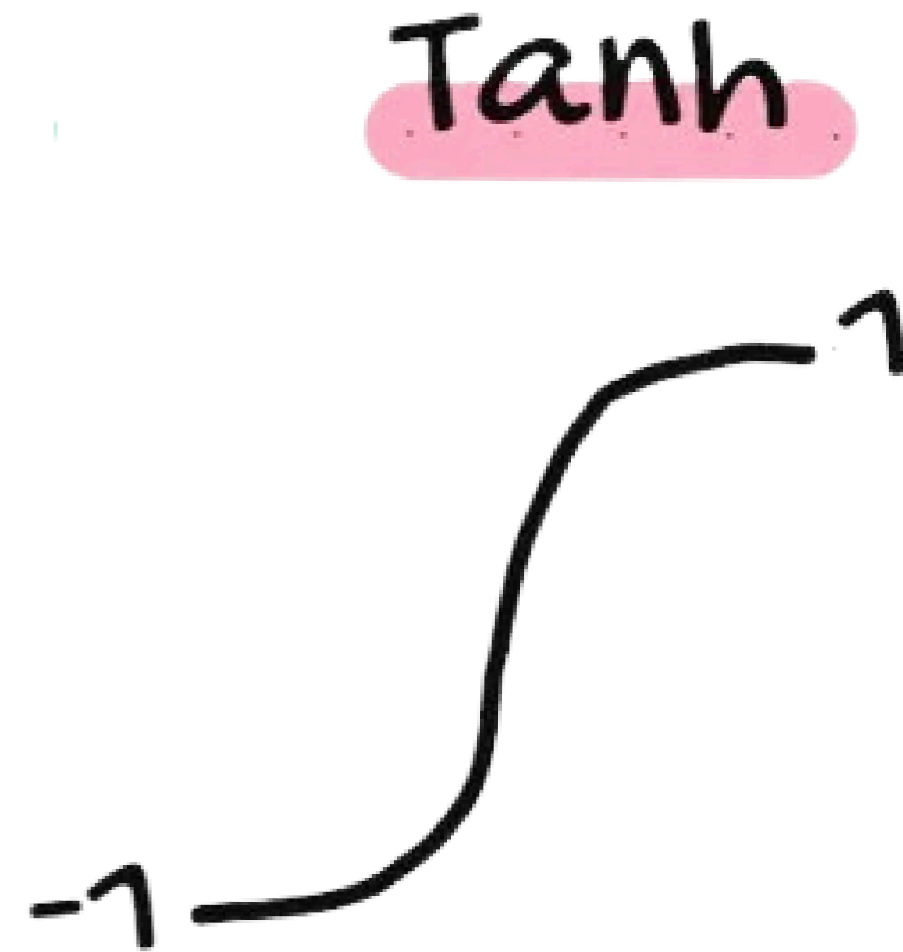


TANH OR HYPERBOLIC TANGENT FUNCTION

หน้าตาของมันจะคล้ายกับ Sigmoid แต่ output ของมันคือ (-1 จนถึง 1) ซึ่ง sigmoid คือ (0 จนถึง 1)

จุดเด่นของฟังก์ชันนี้คือถ้าค่าที่ได้ออกมาเป็นค่าติดลบก็จะมีแนวโน้มไป maps กับค่าที่เป็นลบสูง โดยถ้าค่าเป็น 0 ก็จะมีแนวโน้มไป maps กับค่าใกล้ๆ 0 ในกราฟสูงมาก

tanh function ถูกนิยมนำไปใช้กับ classification ที่มี 2 คลาส



$$\text{Tanh} = (\text{Sigmoid} \times 2) - 1$$

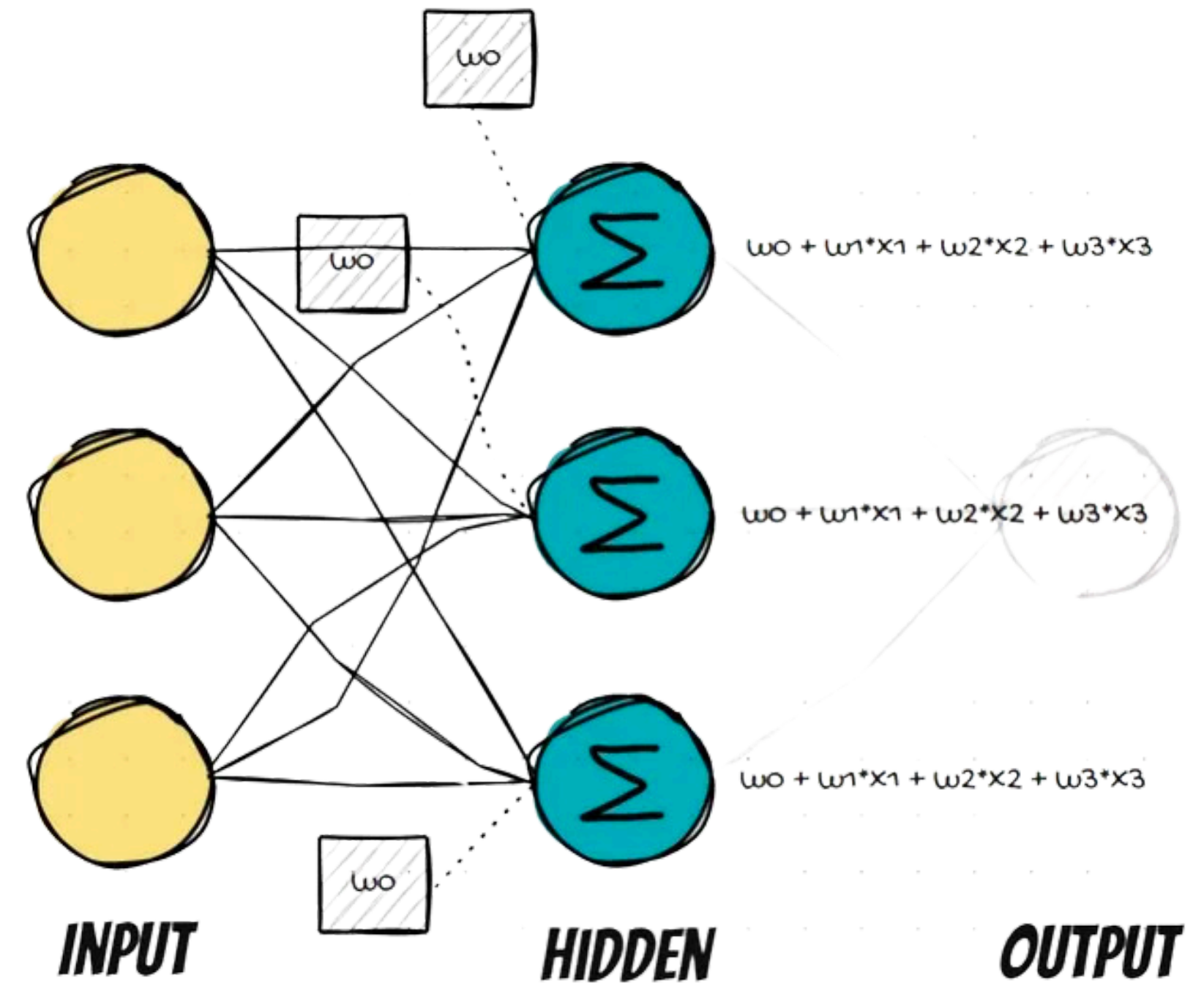
RELU FUNCTION

ReLU ย่อมาจาก Rectified Linear Unit คือ ฟังก์ชันเส้นตรงที่ถูกปรับแก้ Rectified ไม่ได้เป็นรูปตัว S เหมือน 2 ฟังก์ชันก่อนหน้านี้ ReLU เป็นฟังก์ชันที่เรียบง่ายกว่าทุก Activation Function ที่ผ่านมา แต่ตรงพล้ง เนื่องจาก ถ้า Input เป็นบวก Slope จะเท่ากับ 1 ตลอดเวลา ทำให้ Gradient ไม่หาย (ไม่เกิด Vanishing Gradient) ส่งผลให้เรา ทรนโมเดลได้เร็วขึ้นมาก (0 ถึง infinity)



$$A = \max(0, Z)$$

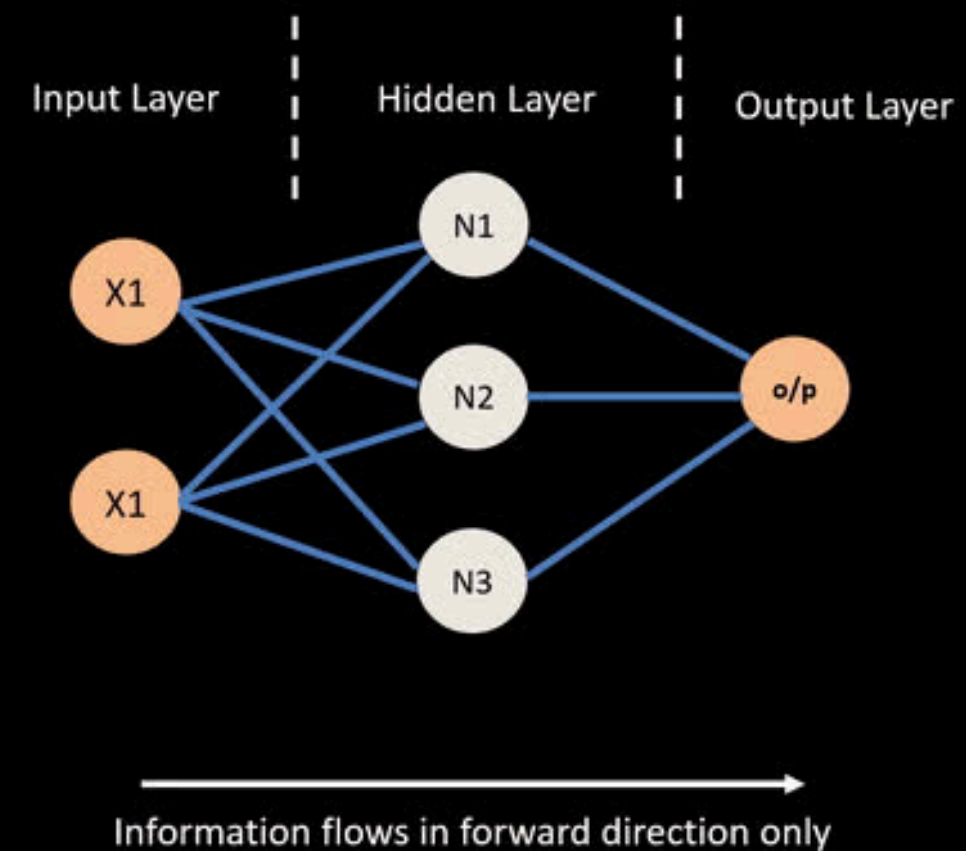
- Input layer ในส่วนของ Input Layer จะมีจำนวน Neuron เท่ากับขนาดของ Data ข้อมูลที่จะส่งเข้ามาใน network
- Hidden layer พอได้ข้อมูลมาแล้ว, แต่ละ Neuron ใน Input Layer ก็จะส่งตัวเลขของ Input ไปให้ Neuron อื่นๆ ใน Layer ต่อไป หรือใน ANN จะเรียกว่า Hidden Layer
- โดยในแต่ละ Neuron ใน Hidden Layer จะมีสิ่งที่เรียกว่า Activation Function เพื่อ Transform ค่าทุกค่า ที่ได้รับมาจากทุกๆ Node ใน Layer ก่อนหน้า เพื่อ ส่งต่อไปยัง Layer ต่อไป,
- โดย Weight ตอนเริ่มต้นของ แต่ละ Input จะเป็นแบบ สุ่ม, และค่อยๆเปลี่ยนแปลงไปเรื่อยๆ เพื่อให้ Fit กับ Data
- พอถ่วงน้ำหนักเรียบร้อย, จะนำแต่ละค่าที่ถูกถ่วงน้ำหนักมา รวมกัน(Sum) และเข้า Activation Function เพื่อ Transform ให้ออกมาเป็นค่า Output และส่งไปยัง Neuron ต่อไป



FORWARD PROPAGATION

สมมุติว่า Algorithm มี Parameter w และ b ที่เป็นตัวแทนของข้อมูลเรียบร้อยแล้ว กระบวนการ Forward propagation คือ การนำข้อมูล x เข้ามาประมวลผลร่วมกับ Parameter เหล่านั้นเป็นชั้นๆ จนได้คำตอบ

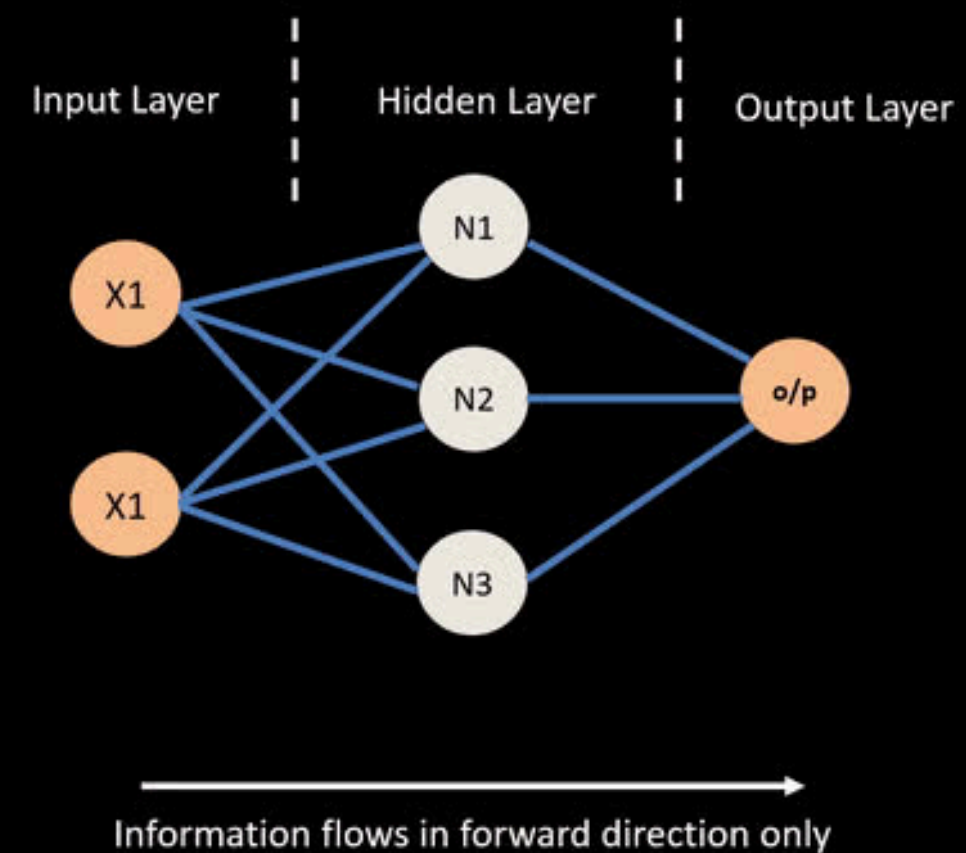
Feed Forward Neural Network



FORWARD PROPAGATION

1. Random $w_1, b_1 \dots w_n, b_n$
2. $A_0 = X$
3. คำนวณ $Z_1 = (wA_0) + b$
4. คำนวณ A_1 จาก activation function $A_1 = g(Z_1)$
5. คำนวณ $Z_2 = (wA_1) + b$
6. คำนวณ A_2 จาก activation function $A_2 = g(Z_2)$
7. ทำไปเรื่อยๆ จนถึง output layer

Feed Forward Neural Network



FORWARD PROPAGATION

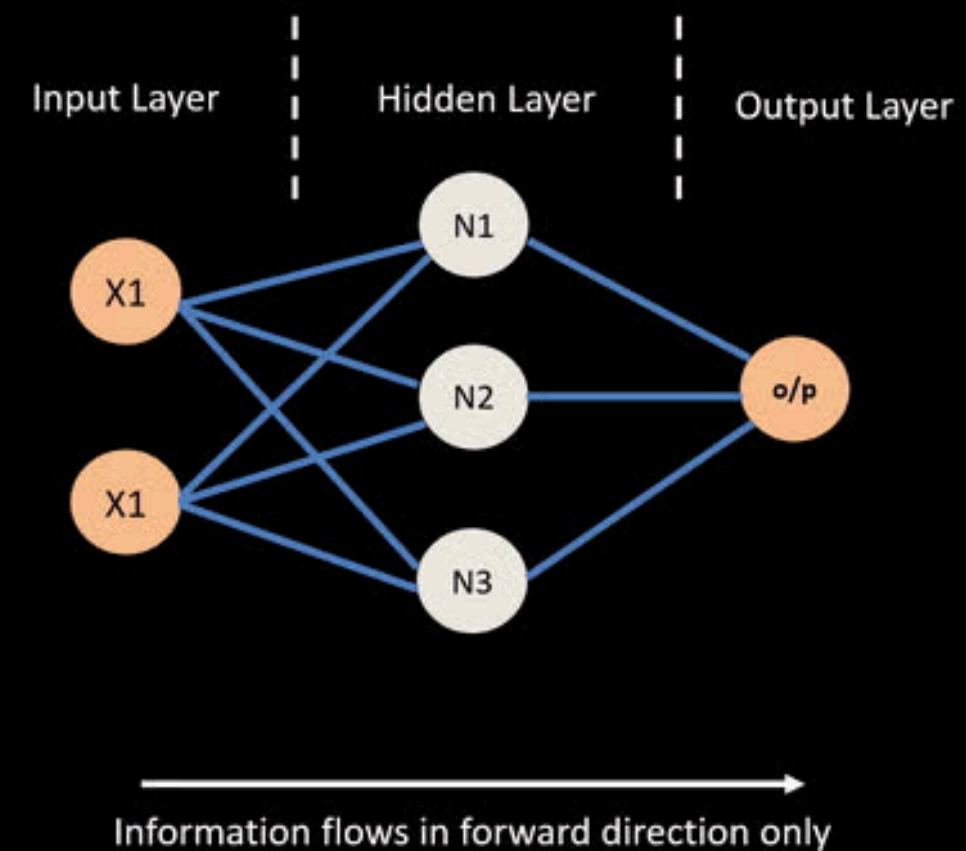
1. Random $w_1, b_1 \dots w_n, b_n$



```
1 def __init__(self):
2     self.W1 = np.random.rand(10, 784) - 0.5
3     self.b1 = np.random.rand(10, 1) - 0.5
4     self.W2 = np.random.rand(10, 10) - 0.5
5     self.b2 = np.random.rand(10, 1) - 0.5
```

Note : ขนาดของ W จะเป็น (in, out) ขนาดของ b จะเป็น (out,) สำหรับการใช้ทั่วไปเช่นใน Tensorflow แต่ถ้าเอามาคำนวณ Matrix แบบนี้ต้องสลับ W เป็น (out, in) b (out,) และต้องปรับ Matrix ให้สัมพันธ์กับ input layer ของเรา

Feed Forward Neural Network



FORWARD PROPAGATION

1. $A0=X$
2. คำนวณ $Z1=(wA0)+b$
3. คำนวณ $A1$ จาก activation function $A1=g(Z1)$
4. คำนวณ $Z2=(wA1)+b$
5. คำนวณ $A2$ จาก activation function $A2=g(Z2)$

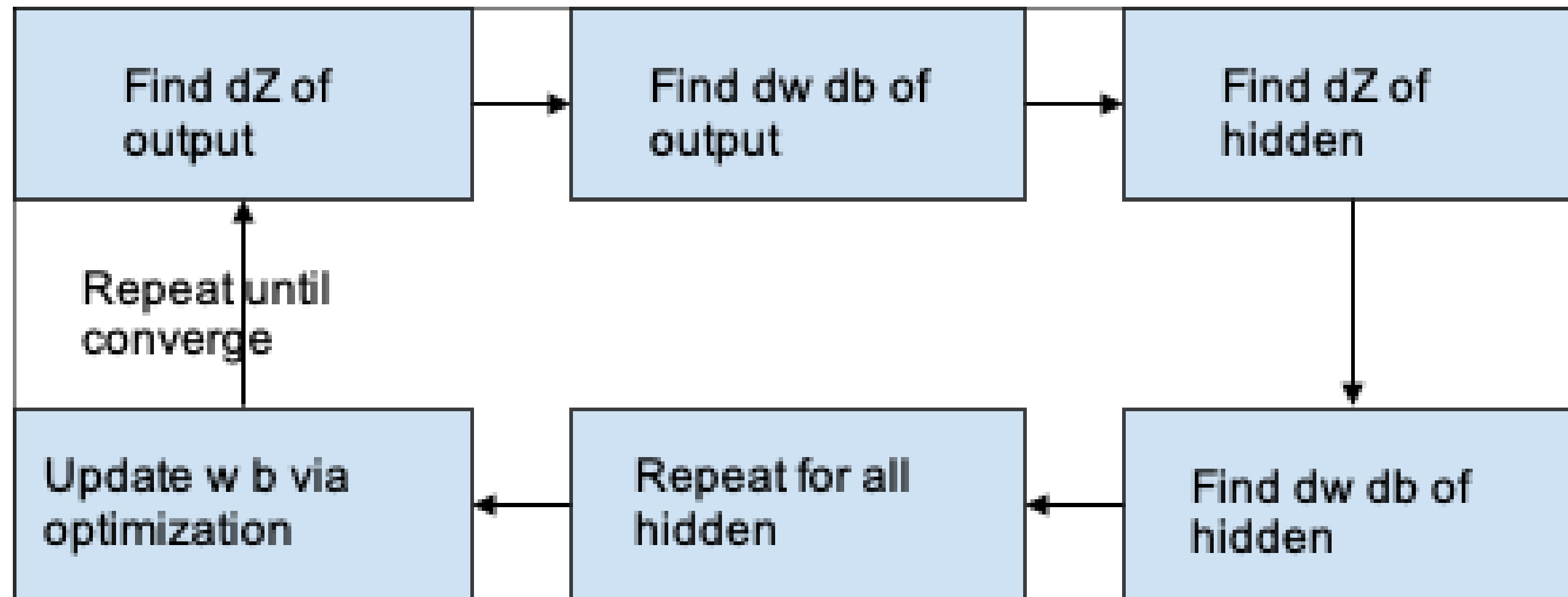


```
1 def forward_prop(self, X):
2     Z1 = np.dot(self.W1, X) + self.b1
3     A1 = relu(Z1)
4     Z2 = np.dot(self.W2, A1) + self.b2
5     A2 = softmax(Z2)
6     return Z1, A1, Z2, A2
```



```
1 def relu(z):
2     return np.maximum(0, z)
3
4 def deriv_relu(z):
5     return z > 0
6
7 def softmax(z): # stable softmax
8     z = z - np.max(z, axis=0)
9     return np.exp(z) / np.sum(np.exp(z), axis=0)
```

BACK PROPAGATION

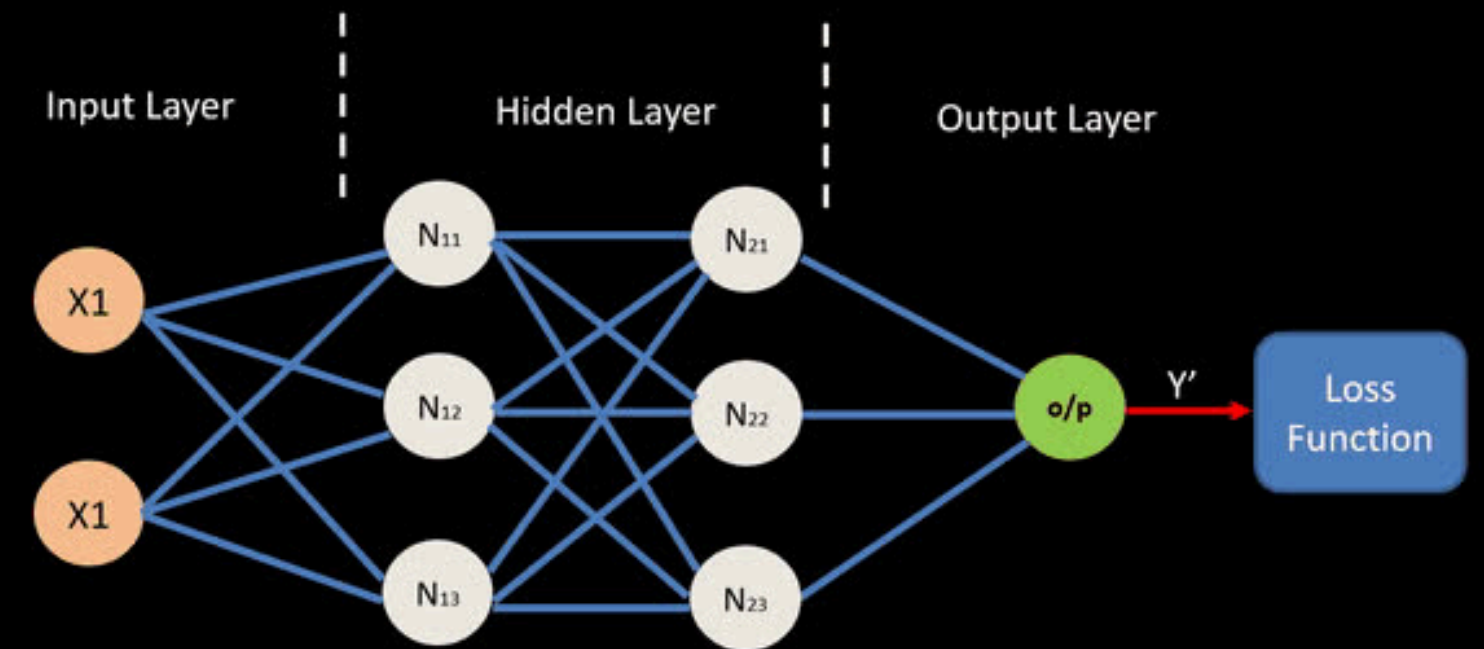


BACK PROPAGATION

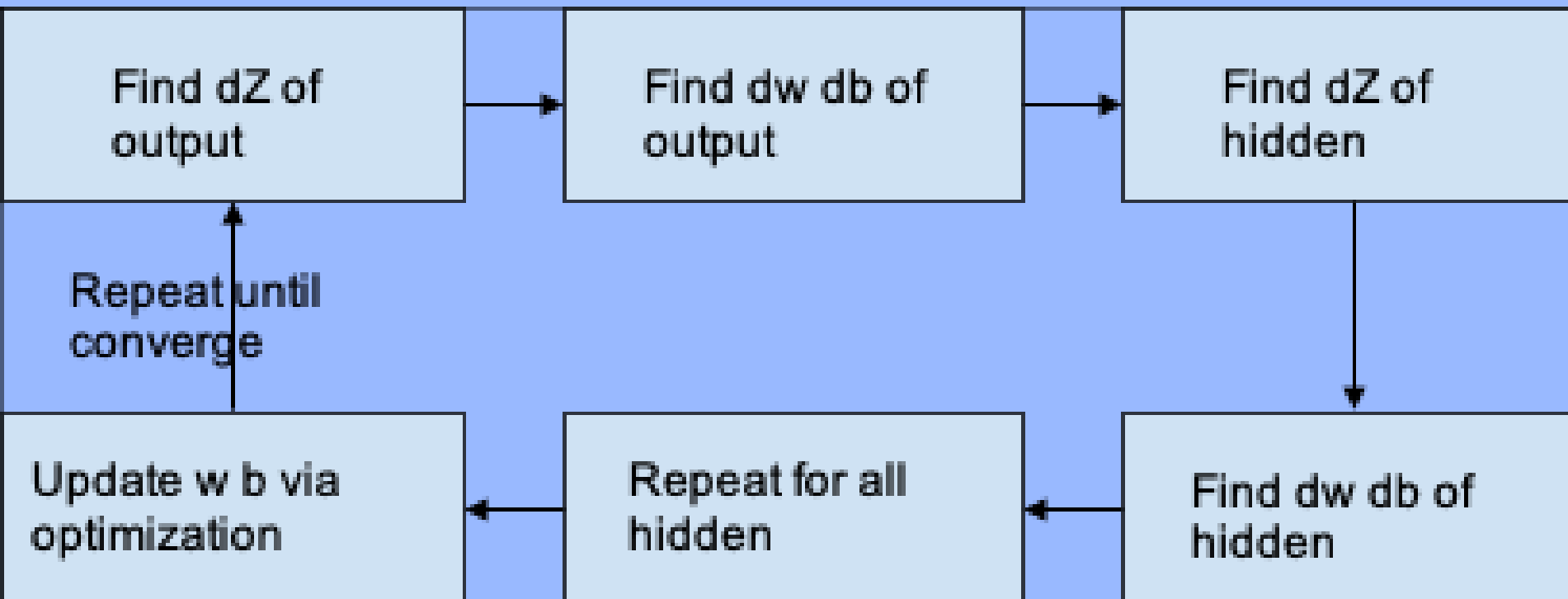
คือส่วนที่ซับซ้อนและยากที่สุดของ Neural network algorithm ซึ่งอาจจะจริงในส่วนของการคำนวณ

ทบทวนว่า กระบวนการ Forward propagation จบลงที่เราได้ค่าพยากรณ์ ดังนั้นสิ่งต่อไปที่เราต้องทำ คือการนำค่าพยากรณ์นั้นมาใส่ใน Cost function เพื่อหาความต่างระหว่างค่าพยากรณ์กับค่าจริง

Neural Network – Backpropagation



BACK PROPAGATION



- คำนวณ dZ ของ output
 - ถ้า output ใช้ softmax *สำคัญ y ต้อง one-hot encoded

$$dZ_{out} = (A_{out} - y)$$

- ถ้า output ใช้อันอื่น $dZ_{out} = (A_{out} - y) * g'(Z^{out})$
- คำนวณ dw ของ output $\frac{dL}{dw_{out}} = \frac{1}{m} (dZ_{out} \cdot A_{hidden}^T)$
 - คำนวณ db ของ output $\frac{dL}{db_{out}} = \frac{1}{m} \sum_{i=0}^{m-1} dZ_{out}$ or mean of dz
 - คำนวณ dZ ของ hidden layer $dZ_{hidden} = (W_{out}^T \cdot dZ_{out}) * g'(Z^{hidden})$
 - คำนวณ dw ของ hidden $\frac{dL}{dw_{hidden}} = \frac{1}{m} (dZ_{hidden} \cdot X^T)$
 - คำนวณ db ของ hidden $\frac{dL}{db_{hidden}} = \frac{1}{m} \sum_{i=0}^{m-1} dZ_{hidden}$ or mean of dz
 - ถ้ามี hidden layer > 1 ก็คำนวณข้อ 4 - 6 จนหมด

- Update w , b via gradient descent

$$W_{out} = W_{out} - \alpha(dw_{out})$$

$$b_{out} = b_{out} - \alpha(db_{out})$$

$$W_{hidden} = W_{hidden} - \alpha(dw_{hidden})$$

$$b_{hidden} = b_{hidden} - \alpha(db_{hidden})$$

- Loop until convergence

$g' = \frac{dA}{dZ}$ g ไพร้ม ขึ้นอยู่กับ activation function ที่ใช้ใน layer นั้นๆ

- Sigmoid Activation Function ReLU (Rectified Linear Unit) Activation Function

$$\frac{dA}{dZ} = A(1 - A)$$

$$\frac{dA}{dZ} = Z > 0$$

BACK PROPAGATION

1. คำนวณ dZ ของ output

a. ถ้า output ใช้ softmax *สำคัญ y ต้อง one-hot encoded

$$dZ_{out} = (A_{out} - y)$$

b. ถ้า output ใช้อันอื่น $dZ_{out} = (A_{out} - y) * g'(Z^{out})$

2. คำนวณ dw ของ output $\frac{dL}{dw_{out}} = \frac{1}{m} (dZ_{out} \cdot A_{hidden}^T)$

3. คำนวณ db ของ output $\frac{dL}{db_{out}} = \frac{1}{m} \sum_{i=0}^{m-1} dZ_{out}$ or mean of dz

4. คำนวณ dZ ของ hidden layer $dZ_{hidden} = (W_{out}^T \cdot dZ_{out}) * g'(Z^{hidden})$

5. คำนวณ dw ของ hidden $\frac{dL}{dw_{hidden}} = \frac{1}{m} (dZ_{hidden} \cdot X^T)$

6. คำนวณ db ของ hidden $\frac{dL}{db_{hidden}} = \frac{1}{m} \sum_{i=0}^{m-1} dZ_{hidden}$ or mean of dz

7. ถ้ามี hidden layer > 1 ก็คำนวณข้อ 4 - 6 จนหมด

8. Update w , b via gradient descent

$$W_{out} = W_{out} - \alpha(dw_{out})$$

$$b_{out} = b_{out} - \alpha(db_{out})$$

$$W_{hidden} = W_{hidden} - \alpha(dw_{hidden})$$

$$b_{hidden} = b_{hidden} - \alpha(db_{hidden})$$

9. Loop until convergence

$g' = \frac{dA}{dZ}$ g ไพรม์ ขึ้นอยู่กับ activation function ที่ใช้ใน layer นั้นๆ

- Sigmoid Activation Function ReLU (Rectified Linear Unit) Activation Function

$$\frac{dA}{dZ} = A(1 - A)$$

$$\frac{dA}{dZ} = Z > 0$$



```
1 def back_prop(self, Z1, A1, Z2, A2, X, Y, alpha):
2     m = len(Y)
3     dZ2 = A2 - one_hot(Y)
4     dW2 = 1/m * np.dot(dZ2, A1.T)
5     db2 = 1/m * np.sum(dZ2)
6     dZ1 = np.dot(self.W2.T, dZ2) * deriv_relu(Z1)
7     dW1 = 1/m * np.dot(dZ1, X.T)
8     db1 = 1/m * np.sum(dZ1)
9
10    self.W2 = self.W2 - alpha * dW2
11    self.b2 = self.b2 - alpha * db2
12    self.W1 = self.W1 - alpha * dW1
13    self.b1 = self.b1 - alpha * db1
```



```
1 def one_hot(Y):
2     m = len(Y)
3     one_hot = np.zeros((10, m))
4     for i in range(m):
5         one_hot[Y[i]][i] = 1
6     return one_hot
```


BACK PROPAGATION

1. คำนวณ dZ ของ output

a. ถ้า output ใช้ softmax *สำคัญ y ต้อง one-hot encoded

$$dZ_{out} = (A_{out} - y)$$

b. ถ้า output ใช้อันอื่น $dZ_{out} = (A_{out} - y) * g'(Z^{out})$

2. คำนวณ dw ของ output $\frac{dL}{dw_{out}} = \frac{1}{m} (dZ_{out} \cdot A_{hidden}^T)$

3. คำนวณ db ของ output $\frac{dL}{db_{out}} = \frac{1}{m} \sum_{i=0}^{m-1} dZ_{out}$ or mean of dz

4. คำนวณ dZ ของ hidden layer $dZ_{hidden} = (W_{out}^T \cdot dZ_{out}) * g'(Z^{hidden})$

5. คำนวณ dw ของ hidden $\frac{dL}{dw_{hidden}} = \frac{1}{m} (dZ_{hidden} \cdot X^T)$

6. คำนวณ db ของ hidden $\frac{dL}{db_{hidden}} = \frac{1}{m} \sum_{i=0}^{m-1} dZ_{hidden}$ or mean of dz

7. ถ้ามี hidden layer > 1 ก็คำนวณข้อ 4 - 6 จนหมด

8. Update w , b via gradient descent

$$W_{out} = W_{out} - \alpha(dw_{out})$$

$$b_{out} = b_{out} - \alpha(db_{out})$$

$$W_{hidden} = W_{hidden} - \alpha(dw_{hidden})$$

$$b_{hidden} = b_{hidden} - \alpha(db_{hidden})$$

9. Loop until convergence

$g' = \frac{dA}{dZ}$ g ไพรม์ ขึ้นอยู่กับ activation function ที่ใช้ใน layer นั้นๆ

- Sigmoid Activation Function ReLU (Rectified Linear Unit) Activation Function

$$\frac{dA}{dZ} = A(1 - A)$$

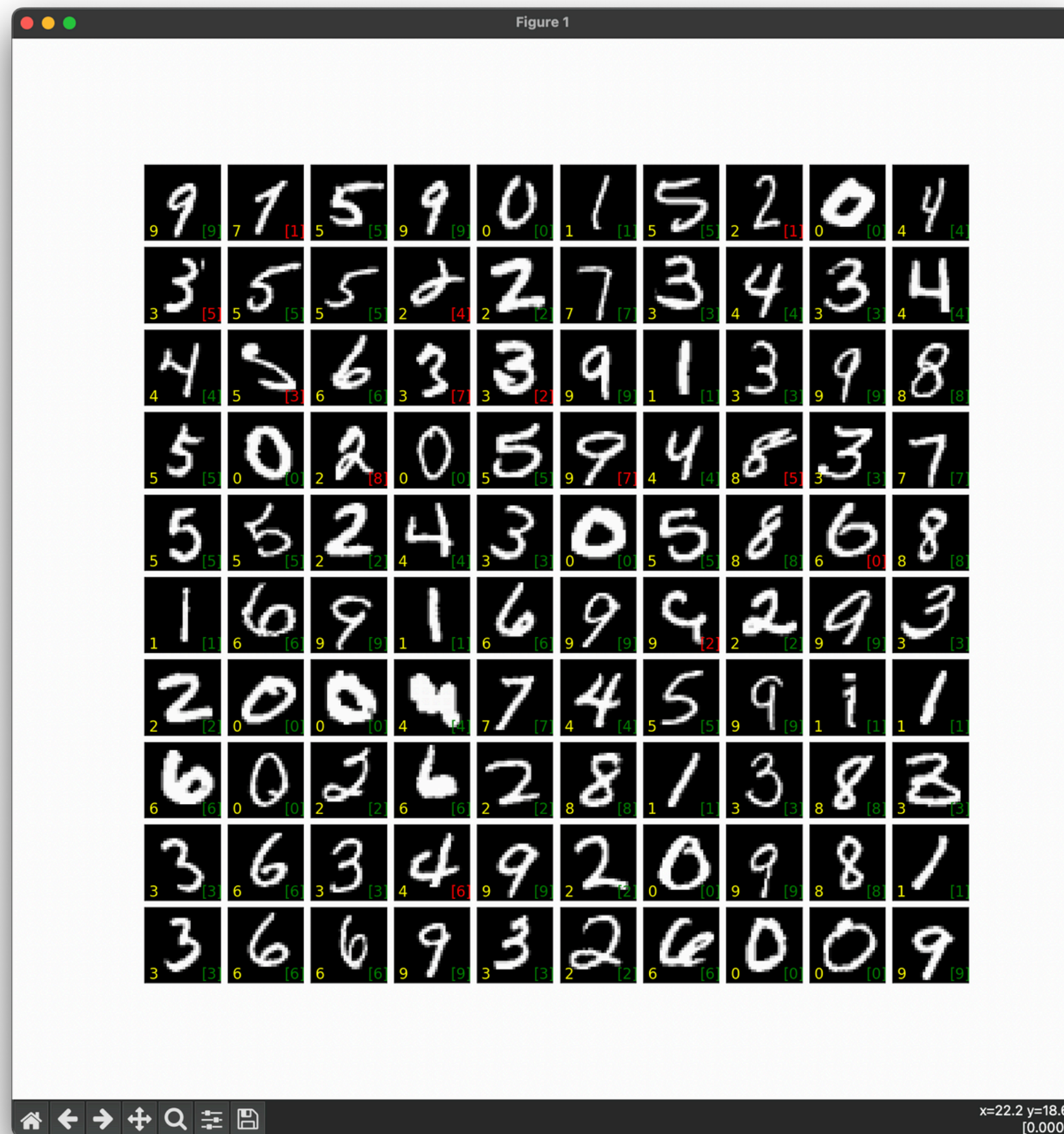
$$\frac{dA}{dZ} = Z > 0$$



```
1 def fit(self, X, Y, alpha, n_iters):
2     for i in range(1, n_iters+1):
3         Z1, A1, Z2, A2 = self.forward_prop(X)
4         self.back_prop(Z1, A1, Z2, A2, X, Y, alpha)
5         accuracy = get_accuracy(A2, Y)
6         if i%100==0 or i==n_iters:
7             print(f'Iteration: {i} Accuracy: {accuracy}')
```



```
1 def get_accuracy(predict, Y):
2     return np.sum(get_predict(predict) == Y) / len(Y)
```



ACCURACY
= 86 %